

Information and Software Technology

ConAgent: Agent-based Concolic Testing to Generate Test Data for Qt Projects --Manuscript Draft--

Manuscript Number:	
Article Type:	Research paper
Keywords:	Automated test data generation; concolic testing; symbolic execution; AI agent; Qt Framework
Corresponding Author:	Duc Anh Nguyen VNU University of Engineering and Technology Faculty of Information Technology VIET NAM
First Author:	Dat Nguyen Trong
Order of Authors:	Dat Nguyen Trong
	Duc Nguyen Huu
	Linh Ngoc Truc Tran
	Duc-Anh Nguyen
	Pham Ngoc Hung
Abstract:	Context: Qt Framework is one of the most popular frameworks for developing cross-platform applications. Concolic testing is widely used to generate test data for such applications. Recently, using LLM to generate test data is a novel approach. Statement coverage is a common metric to generate test data. Objectives: Concolic testing does not support Qt project effectively, due to the problem of Qt object initialization, path constraint solving, and stub usage. The LLM approach deals with the limitations of LLMs such as inefficient LLM inputs, unreliable internet data sources, and LLM hallucinations. As a result, achieving high statement coverage is challenging. Therefore, this paper proposes ConAgent to mitigate these problems. Methods: The general idea is to combine concolic testing and AI agents to generate test data to maximize statement coverage. Briefly, ConAgent consists of three phases, including agent-based initialization phase, agent-based simplification phase, and multi-agent phase. The agent-based initialization phase constructs an initialization prompt, then uses agent generator to generate a minimum set of test data. For each test path covering unvisited statements, the agent-based simplification phase creates a simplification prompt and applies agent generator generate the next test data. If the agent-based simplification phase fails, the multi-agent phase combines agent generator and agent validator to generate test data more effectively. Results: Experiments are conducted on three real projects on GitHub. The comparable method is UT4UQ, the first concolic testing method for Qt project. In terms of average statement coverage, while UT4UQ achieves 46.5%, the proposed method reaches up to above 70% for both DeepSeek 32B and GPT-4o. Additionally, the proposed method could deploy locally to ensure data privacy by using pretrained models such as DeepSeek 32B, while achieving the approximate statement coverage as GPT-4o. Conclusion: The experimental results demonstrate the effectiveness of ConAgent in practical applications.

Highlights

CONAGENT: Agent-based Concolic Testing to Generate Test Data for Qt Projects

Dat Nguyen Trong, Duc Nguyen Huu, Linh Ngoc Truc Tran, Duc Anh Nguyen, Pham Ngoc Hung

- **First** agent-based concolic testing approach for Qt Framework projects
- Solves Qt object initialization, path constraint solving, stub usage, and LLM limitation issues
- Apply RAG to enhance the prompt to LLMs

CONAGENT: Agent-based Concolic Testing to Generate Test Data for Qt Projects

Dat Nguyen Trong, Duc Nguyen Huu, Linh Ngoc Truc Tran, Duc Anh Nguyen*, Pham Ngoc Hung

VNU University of Engineering and Technology (VNU-UET)
E3 building, 144 Xuan Thuy st., Cau Giay dist., Hanoi, Vietnam

Abstract

Context: Qt Framework is one of the most popular frameworks for developing cross-platform applications. Concolic testing is widely used to generate test data for such applications. Recently, using LLM to generate test data is a novel approach. Statement coverage is a common metric to generate test data.

Objectives: Concolic testing does not support Qt project effectively, due to the problem of *Qt object initialization*, *path constraint solving*, and *stub usage*. The LLM approach deals with *the limitations of LLMs* such as inefficient LLM inputs, unreliable internet data sources, and LLM hallucinations. As a result, achieving high statement coverage is challenging. Therefore, this paper proposes **CONAGENT** to mitigate these problems.

Methods: The general idea is to combine concolic testing and AI agents to generate test data to maximize statement coverage. Briefly, **CONAGENT** consists of three phases, including agent-based initialization phase, agent-based simplification phase, and multi-agent phase. The agent-based initialization phase constructs an initialization prompt, then uses agent generator to generate a minimum set of test data. For each test path covering unvisited statements, the agent-based simplification phase creates a simplification prompt and applies agent generator generate the next test data. If the agent-based simplification phase fails, the multi-agent phase combines agent generator and agent validator to generate test data more effectively.

Results: Experiments are conducted on three real projects on GitHub. The comparable method is UT4UQ, the first concolic testing method for Qt project. In terms of average statement coverage, while UT4UQ achieves 46.5%, the proposed method reaches up to above 70% for both DeepSeek 32B and GPT-4o. Additionally, the proposed method could deploy locally to ensure data privacy by using pretrained models such as DeepSeek 32B, while achieving the approximate statement coverage as GPT-4o.

Conclusion: The experimental results demonstrate the effectiveness of **CONAGENT** in practical applications.

Keywords: Automated test data generation, concolic testing, symbolic execution, AI agent, Qt Framework.

*Corresponding author

Email addresses: 24025033@vnu.edu.vn (Dat Nguyen Trong), 22028184@vnu.edu.vn (Duc Nguyen Huu), linhhtnt@vnu.edu.vn (Linh Ngoc Truc Tran), nguyenducanh@vnu.edu.vn (Duc Anh Nguyen), hungpn@vnu.edu.vn (Pham Ngoc Hung)

Preprint submitted to Information and Software Technology

June 18, 2025

1. Introduction

Qt Framework is one of the most common frameworks for developing cross-platform applications. This framework supports writing backend and frontend in C++. According to Qt Company estimates in 2017, over one million developers in more than 70 countries [1, 2] used this framework in their projects. To ensure the quality of Qt projects, unit testing should be performed intensively. It is a topic of great interest among researchers and companies. The main objective of unit testing is to ensure the quality of functions. In this context, a tested function is called *focal method*.

There are two main approaches of unit testing including black-box approach [3, 4] and white-box approach [5, 6]. While black-box approach does not consider the internal structure of Qt projects, white-box approach analyzes source code to detect faults. As a result, the usage of white-box approach would find out potential faults inside focal methods better than using black-box approach. Therefore, white-box approach is commonly used to evaluate the quality of highly critical applications such as autonomous systems, drones, etc. The input of white-box approach is a focal method and a coverage metric. The output is a set of test data which aims to maximize the coverage metric. The most common coverage metric is statement coverage. It is expected that all statements of the focal method are executed given a set of test data.

However, the main drawback of the white-box approach is its high computational cost [7], especially when Qt projects may contain a large number of focal methods. The effort required to manually create and maintain test data results in significant costs throughout the development and maintenance cycles. Therefore, white-box approach should be automated to save time and reduce human error. Unfortunately, most available testing tools such as Valgrind [8], Kcov [9], VectorCAST [10] lack compatibility with the features of Qt, limiting their effectiveness in real-world Qt projects.

Since its inception, concolic testing has been a standout automated white-box testing solution with the potential to ensure quality [11, 12, 13, 14, 15, 16, 17, 18]. The main idea of concolic testing is to combine static analysis and dynamic analysis to generate high-quality test data. Firstly, from a focal method and a coverage metric such as statement coverage, a random test data is generated. The test data is then executed to get visited statements, which form a test path. Concolic testing negates the test path to generate a new test path, which contains unvisited statements. Next, the new test path is symbolized to generate a path constraint. The path constraint is solved by using an SMT-Solver such as Z3 [19], SmtInterpol [20], etc. The solution corresponds to a new test data. It is expected that the execution of the new test data will go through the new test path, which helps to increase the coverage. This process repeats until all statements are covered.

Although numerous studies on applying concolic testing [13, 21, 22, 23, 24, 25, 26] have been published, there is less research about the application of concolic testing to Qt projects. Existing automated test data generation strategies [27, 28, 29, 30] have shown limited effectiveness when applied to complex data types, such as those used in the Qt Framework. To the best of our knowledge, UT4UQ [31] is the first method to generate test data for Qt projects. When applied to Qt projects, there are several main issues including *Qt object initialization*, *path constraint solving*, and *stub usage*.

Firstly, the problem of *Qt object initialization* refers to ability to generate the initial test data. Concolic testing usually applies random techniques to generate the initial test data to start the testing process [32, 33, 34]. An initial test data is high-quality if the execution of the test data goes through as many statements as possible. To initialize a Qt object, a list of relevant Qt constructors should be identified at first. There are two main types of constructors including

parameterized constructors and no-argument constructors. After obtaining the list of constructors, one is selected, and the parameters of this constructor are then initialized. If a parameter is Qt class parameter, its constructors are then collected. Generally, the process of Qt object initialization is recursive until there is no Qt class parameter, but only parameters of primitive data types remain to be randomly assigned values. A naive solution to this problem is to select no-argument constructors. The main reason is that no-argument constructors do not contain any parameters, which could avoid the recursive problem. However, high-quality test data could require parameterized constructors. Therefore, the usage of only no-argument constructors could ignore high-quality test data.

Secondly, concolic testing faces the problem of *path constraint solving*. A path constraint is the result of applying symbolic execution on a test path [35]. A path constraint consists of constraints, which are usually symbolized branch conditions. A branch condition is symbolized by replacing local variables with expressions of parameters and global variables. For example, consider a simple test path of a focal method as follows: $\text{int example}(\text{int } a, \text{int } b) \rightarrow \text{int } c = a + f(b) \rightarrow (c > 0) \text{ is True} \rightarrow \text{return } c$. By applying symbolic execution, the branch condition $c > 0$ is symbolized into an expression $a + f(b) > 0$, in which a and b are parameters. The solution of the path constraint is expected to go through the test path. Current SMT solvers, such as Z3, does not support derived types. This means that if a path constraint contains Qt objects, SMT solvers are unable to solve it. As a result, the next test data is not generated, which could lead to low statement coverage. Therefore, it is necessary to find a solution to generate test data for such path constraint.

Finally, a major challenge that arises during concolic testing is *stub usage*. During constraint solving, one important task is to handle the invoked function. An invoked function is a function called by the focal method. Stubbing is often used to assume the return value of the invoked function [36], since solving the constraints within these invoked functions to satisfy the constraints in the focal method can be highly complex. For example, in the above test path, the invoked function $f()$ is stubbed to return 2. The expression $c = a + f(b)$ turns into $c = a + 2$, which is simpler to solve by SMT solvers. One approach to automatically determine the stub values is to use SMT solvers. SMT solvers treat $f(b)$ as a variable and attempt to find a suitable value for this variable. However, the stub value generated by SMT solvers for the invoked function $f(b)$ may never actually be producible in practice. Even though the generated test data may cover a specific test path, this test path would not be feasible in real execution because the invoked function $f(b)$ can not return the stub value.

Recently emerging LLM-based techniques have shown promise in supporting concolic testing. The main idea is to generate a prompt, and LLM is invoked to generate test data from the prompt. However, their practical application remains limited due to *the limitations of LLMs*. This limitations includes inefficient LLM inputs, unreliable internet data sources, and LLM hallucinations. First of all, optimizing LLM for test data generation requires a well-structured context input. Rosas et al. [37] note that LLM accuracy declines with larger prompts. Several recent studies [38, 39, 40, 41] have focused on optimizing inputs for LLMs. Despite these advancements, existing methods focus on uncovered code rather than analyzing test paths, requiring LLMs to process multiple paths while filtering irrelevant statements. Even with optimized input, generating advanced test data with LLM remains difficult. Unlike Z3, LLM lacks inherent accuracy and stability, relying on publicly available data that can introduce errors. Despite Qt's extensive documentation, retrieving accurate data remains challenging. LLM hallucination further exacerbates these issues, as errors in one iteration can propagate and degrade test data quality in subsequent attempts.

To address the aforementioned issues, this research proposes a method called **CONAGENT** (**Agent-based Concolic Testing**). To the best of our knowledge, this is the **first** agent-based concolic testing for Qt projects. The input is a Qt project. For each focal method in this project, the output is a test report including a set of test data and statement coverage. Generally, **CONAGENT** consists of three consecutive main phases, including agent-based initialization phase, agent-based simplification phase, and multi-agent phase. The agent-based initialization phase generates an initialization prompt, then uses agent generator to generate a minimum set of initial test data to maximize statement coverage. The initialization prompt is designed to overcome the problem of *Qt object initialization*. For each test path containing unvisited statements, the agent-based simplification phase generates a simplification prompt, then uses agent generator to enhance the statement coverage. The simplification prompt addresses the problem of *path constraint solving* and *stub usage*. If the agent-based simplification phase fails, the multi-agent phase employs agent generator and agent validator to generate test data more effectively. The usage of these two agents degrades *the limitations of LLMs* to generate test data.

Experiments show that **CONAGENT** achieves promising results when applied to large-scale open-source C++ projects using Qt Framework on GitHub. Specifically, **CONAGENT** produces significantly better coverage than concolic testing. **CONAGENT**, containing two LLM models: GPT-4o and DeepSeek 32B (short term for DeepSeek R1 Distill Qwen 32B), achieves a higher average statement coverage of 72% and 70.7% respectively, surpassing UT4UQ (46.5%) about 25%. Furthermore, leveraging open-source, cost-effective LLMs not only improves statement coverage but also optimizes computational efficiency and cost, making **CONAGENT** scalable for larger projects.

Briefly, **CONAGENT** has the following key contributions:

- Propose a three-phase method to generate a set of test data. The agent-based initialization phase generates a minimum set of initial test data, which addressing the problem of *Qt object initialization*. The agent-based simplification phase addresses the problem of *path constraint solving* and *stub usage*. The multi-agent phase combines concolic testing and agents to solve the problem of *the limitations of LLMs*.
- The demonstration of the effectiveness of open-source models, offering a cost-saving solution that can be feasibly scaled for larger projects.

The remainder of this paper is organized as follows. Section 2 presents the background knowledge. The challenges encountered when applying concolic testing to generate test data for Qt projects are demonstrated through the example presented in Section 3. Section 4 presents a detailed explanation of the method. Section 5 describes the experimental evaluation conducted on several Qt projects and compares the results with those obtained from existing approaches. Section 6 reviews related work and highlights their limitations in handling complex libraries like Qt. Finally, Section 7 concludes the paper and suggests directions for future research.

2. Background

2.1. Qt Class

Qt classes are the fundamental building blocks of applications developed using the Qt Framework. Qt classes are diverse, as they encompass a wide range of objects with distinct characteristics. All Qt classes are implemented in C++. Many of them inherit, directly or indirectly, from

the base class `QObject`, which grants them access to Qt’s powerful meta-object system, signals and slots mechanism, properties, event handling, and internationalization features. Key characteristics of Qt classes, particularly those inheriting from `QObject`, relevant to automated testing include:

- *Signals and Slots*: This mechanism allows for decoupled communication between objects. An object can emit a signal when its state changes, and other objects can connect slots (member functions) to these signals to react. This leads to event-driven control flow, which can be challenging for path exploration techniques like concolic testing to follow and model accurately in path constraints.
- *Complex Data Types*: Qt Framework offers a rich set of non-primitive data types beyond basic C++ types, such as `QString`, `QList`, `QVector`, `QMap`, `QVariant`, and various GUI-related classes (`QWidget`, `QPixmap`, etc.). These types often have complex internal attributes, specific constructors, and methods that are opaque to standard SMT solvers used in concolic testing.

These features, while enabling robust and feature-rich applications, present significant challenges for automated test generation tools. Generating valid and diverse initial instances of complex Qt class parameter is difficult using simple random techniques. Representing the state and behavior of Qt class parameter and their interactions symbolically for path constraint solving is often infeasible with standard SMT solvers that primarily support primitive types.

2.2. Concolic Testing

Concolic testing, which merges *Concrete* execution with *Symbolic* execution, was first introduced by Larson and Austin [11] in 2003. The term “Concolic” was later coined by Sen et al. [42] in 2005. Concrete execution runs a program with specific input values to observe actual behavior, while symbolic execution analyzes the program with symbolic inputs to explore multiple execution paths and derive input constraints. Algorithm 1 shows the main idea of concolic testing.

The method consists of two main phases including project analyzer and unit test generation. The purpose of the project analyzer phase is to preprocess the source code in preparation for the unit test generation phase. First, the project analyzer phase analyzes source code P to find relationships between source code components, denoted as R (line 2). A source code component includes packages, files, enums, classes, structs, and other elements. For example, if a source file corresponding to component A includes a header corresponding to component B , then there will be a connection describing the relationship between these two components. Next, this phase inserts instrumentation statements into P (line 3). The source code after inserting instrumentation statements is called P_{ins} . These instrumentation statements print out the executed statements and branches. Statement coverage will be calculated based on this information.

After the source code preprocessing is complete, the unit test generation phase extracts all focal methods from the project (line 4). The result set $\mathcal{M}$ is initialized as empty (line 5). The method then processes each focal method fm iteratively.

For each focal method fm , concolic testing generates an initial test data by applying concrete execution denoted by t (line 8). Then, the test data is executed to obtain the test path tp and statement coverage (line 10). However, the achieved statement coverage may not be optimal. Concolic testing negates the executed test path tp to obtain $\overline{tp}$ (line 15). Then, constraints pc are generated by applying symbolic execution on $\overline{tp}$ (line 16). The solution of pc is found out by

Algorithm 1 Concolic Testing

Input: P - a Qt project.

Output: $\mathcal{M}$: set of pairs $\{(fm, T)\}$ where fm is focal method and T is generated test data set

```
1: // Project analyzer
2:  $R \leftarrow \text{AnalyzeProject}(P)$ 
3:  $P_{ins} \leftarrow \text{InsertInstrumentation}(P)$ 
4:  $F \leftarrow \text{ExtractFocalMethods}(P)$ 
5:  $\mathcal{M} \leftarrow \emptyset$ 
6: for  $i$ -th  $fm \in F$  do
7:   // Unit test generation
8:    $t \leftarrow \text{GenInitialTD}(fm)$ 
9:   repeat
10:     $tp, coverage \leftarrow \text{ExecuteTestData}(t, P_{ins})$ 
11:    if  $coverage$  increases then
12:       $T \leftarrow T \cup \{t\}$ 
13:    end if
14:    if  $coverage < 100\%$  then
15:       $\overline{tp} \leftarrow \text{NegatePath}(tp)$ 
16:       $pc \leftarrow \text{SymbolicExecution}(\overline{tp})$ 
17:       $t \leftarrow \text{SMTSolver}(pc)$ 
18:    end if
19:  until  $coverage = 100\%$  or time out
20:   $\mathcal{M} \leftarrow \mathcal{M} \cup \{(fm, T)\}$ 
21: end for
22: return  $\mathcal{M}$ 
```

using SMT-Solvers (line 17). The process terminates when all statements are visited by a set of test data T or time out.

2.3. LLM

A language model (LM) is a collection of prior knowledge about a specific language, including vocabulary, grammar, and the frequency of phrase occurrences. A language model can be constructed using either expert-driven or data-driven approaches. The classic definition of a language model is a probability distribution over sequences of tokens. Given a vocabulary V of tokens, a language model p assigns a probability to each sequence $x_1, \dots, x_L \in V$ as:

$$p(x_1, \dots, x_L) \tag{1}$$

LLMs represent a significant advancement over traditional LMs [43]. Built on transformer-based architectures and trained on massive datasets, LLMs can understand and generate human-like language with impressive fluency and contextual awareness. While traditional LMs often suffer from limited training data and narrow scopes, LLMs capture broader patterns and semantics due to their vast parameter space and data coverage [44]. However, despite their capabilities, LLMs still face challenges when dealing with up-to-date knowledge or domain-specific queries that fall outside the scope of their training data [45]. This is where Retrieval Augmented Generation (RAG) becomes instrumental. RAG is a hybrid architecture that combines two core components: retrieval and generation. It enables LLMs to dynamically get relevant external information at inference time and condition their responses on this retrieved content. By doing so,

RAG makes LLMs significantly more accurate and adaptable, particularly in scenarios requiring real-time or specialized knowledge.

2.4. AI agent

AI agent is an intelligent entity or system that interacts with the environment to achieve a specific goal or solve a problem [46]. An agent can receive the information from the environment in the form of observation, and then use this information to make decisions and take actions [47].

The process of an AI agent typically includes the following repetitive stages:

- Reasoning: The agent uses the available information to make assumptions or judgments about the problem. This process involves analyzing each step of the problem to find a suitable solution, helping the agent make more accurate decisions.
- Action: After each step of reasoning, the agent performs a specific action (e.g., searching for information, or retrieving data). Each action the agent takes may provide additional information or solutions for the next step. In this process, if the agent lacks information, it may call functions (function calling) to retrieve data from external sources or use supporting tools to complete the task.
- Observation: Each action taken will bring feedback from the environment, allowing the agent to appraise whether the action was correct. This result is used to supplement the reasoning process for the next step so the agent can adjust its actions to move towards the ultimate goal.

3. Motivational Example

This section presents a motivational example to show the limitation of concolic testing. Listing 1 shows a typical focal method named *mouseDoubleClickEvent* for GUI handling and its related information. The parameter of this focal method is Qt class parameter.

Listing 1: Example of Qt function in QtQuickExample.

```

1  // Class constructor
2  DDNewQuickWindow::DDNewQuickWindow(QQuickView *parent): QQuickView(parent)
3  {
4      setFlags(Qt::FramelessWindowHint|Qt::WindowStaysOnTopHint);
5      setColor(Qt::transparent);
6      setResizeMode(QQuickView::SizeRootObjectToView);
7      setMinimumWidth(816);
8      mHelper = new DDFramelessHelper(this);
9      mHelper->activateOnQmlWidget(this);
10     mHelper->setTitleHeight(100);
11 }
12 ...
13 // Focal method
14 void DDNewQuickWindow::mouseDoubleClickEvent(QMouseEvent *event) ①
15 {
16     QRect rect(10, 2, this->width() - 70, mHelper->titleHeight()); ②
17
18     if(event->button() == Qt::LeftButton && rect.contains(event->pos())) ③ {
19         ...

```

```

20     }
21     QQuickView::mouseDoubleClickEvent(event); ④
22 }
23 ...

```

The `DDNewQuickWindow` class is designed to create a custom frameless window with specific behaviors such as staying on top and having a transparent background. The focal method `mouseDoubleClickEvent` is responsible for handling mouse double-click interactions. This method aims to toggle the window’s maximization state if a left mouse button double-click occurs within a designated title bar area. The primary condition, `event->button()==Qt::LeftButton && rect.contains(event->pos())` (line 18), evaluates to true if and only if two sub-conditions are met: (i) the event is identified as a left-button click, and (ii) the mouse cursor’s position at the time of the event lies within the calculated rect. Given that the class constructor initializes the window with a minimum width of 816 pixels (`setMinimumWidth(816)`) (line 7) and sets the title height to 100 pixels via the helper object (`mHelper->setTitleHeight(100)`) (line 10), the rect for a window in the focal method should become `QRect(10, 2, 746, 100)`.

Problem of concolic testing: Applying concolic testing to generate such satisfying test data for this Qt example encounters several significant challenges.

Firstly, the initial test data generation faces *Qt object initialization* when trying to select an appropriate `QMouseEvent` constructor. The selection of constructor parameters presents significant difficulties. There is no inherent mechanism to guide the selection towards a semantically relevant constructor for the `mouseDoubleClickEvent` handler. Many `QMouseEvent` parameters, such as event types or button flags, are not primitive types.

Secondly, the directed test data generation faces *path constraint solving*. Consider the test path ① → ② → ③ (True) → ④, the condition ③ (True) is hard to solve by SMT solvers. The main reason is that SMT solvers lack native support for Qt-specific data types such as `QRect` and `QPoint`.

Finally, the *stub usage* also introduces complexities. For Qt methods like `rect.contains()` in ③, it is straightforward to stub its return value (e.g., `true`) to satisfy the above path constraint. However, generating `QRect` and `QEvent` objects such that `rect.contains()` return `true` is naturally evaluates to the desired outcome would yield more realistic test cases. For user-defined methods such as `mHelper->titleHeight()` in ②, concolic analysis might stub it to return a value satisfying a specific path constraint, yet in this example, `titleHeight()` consistently returns 100 due to the `mHelper->setTitleHeight(100)` invocation in the constructor. This divergence can lead to test scenarios achieving coverage for paths that are, in practice, unreachable during actual program execution.

4. CONAGENT

To solve the problem of *Qt object initialization*, *path constraint solving*, *stub usage*, and *the limitations of LLMs*, this research proposes a method named **CONAGENT**. The main idea of **CONAGENT** is to use LLM to generate test data. An overview of **CONAGENT** is illustrated in Algorithm 2. Notion (*) shows the steps identical to concolic testing.

For each focal method *fm*, the agent-based initialization phase *InitializationPhase(.)* generates an initial set of test data by proposing an initialization prompt and using agent generator (line 6). Then, the test data execution step executes this test data to calculate statement coverage

Algorithm 2 Overview of **CONAGENT**. Notion (*) shows the steps identical to concolic testing.

Input: P - a Qt project.

Output: $\mathcal{M}$: set of pairs $\{(fm_i, T)\}$ where fm is focal method and T is generated test data set

```

1:  $R \leftarrow \text{AnalyzeProject}(P)$  ▷ (*)
2:  $P_{ins} \leftarrow \text{InsertInstrumentation}(P)$  ▷ (*)
3:  $F \leftarrow \text{ExtractFocalMethods}(P)$  ▷ (*)
4:  $\mathcal{M} \leftarrow \emptyset$  ▷ (*)
5: for  $i$ -th  $fm \in F$  do ▷ (*)
6:    $T \leftarrow \text{InitializationPhase}(fm)$  ▷ Sect. 4.1
7:    $coverage \leftarrow \text{ExecuteTestData}(T, P_{ins})$  ▷ (*)
8:    $P_{uncov} \leftarrow \text{FindUnvisitedTestPaths}(fm)$ 
9:   for each test path  $tp \in P_{uncov}$  do
10:     $t \leftarrow \text{SimplificationPhase}(fm, tp)$  ▷ Sect. 4.2
11:    if  $t$  is not generated successfully then
12:       $t \leftarrow \text{MultiAgentPhase}(fm, tp)$  ▷ Sect. 4.3
13:    end if
14:     $T \leftarrow T \cup t$  ▷ (*)
15:  end for
16:   $\mathcal{M} \leftarrow \mathcal{M} \cup \{(fm, T)\}$  ▷ (*)
17: end for
18: return  $\mathcal{M}$ 

```

(line 7). However, the achieved statement coverage may not be optimal. For each test path containing unvisited statements (line 8), the agent-based simplification phase *SimplificationPhase*(.) generates a simplification prompt and applies agent generator to generate test data (line 10). The generated test data is expected to traverse this test path. If agent-based simplification phase fails, multi-agent phase *MultiAgentPhase*(.) employs two agents, namely agent generator and agent validator, to generate test data more effectively. Finally, each focal method and its corresponding test data set are collected as pairs in the result set (line 16).

4.1. Agent-based initialization phase

From the focal method fm , to solve the *Qt object initialization*, this phase generates a minimal set of test data to maximize statement coverage. To achieve this goal, this phase creates an initialization prompt and then utilizes an agent generator to produce test data. Figure 1a presents the structure of the proposed initialization prompt. The initialization prompt consists of *backstory* and *prompt*. The part *backstory* is used to set up the behavior of the agent generator, defined in $\{Expert\ profile\}$ and $\{Chain-of-thought\}$. Specifically, the part $\{Expert\ profile\}$ describes the capability of the agent generator. It is expected that the agent generator is designed with profile of a software testing and source code analysis expert, particularly knowledgeable in C++ and the Qt Framework. The agent generator has extensive experience in automated testing, control flow graph analysis, and path constraint solving. Based on the above profile, the agent generator applies Chain-of-thought technique [48] to complete the prompt effectively.

In concolic testing, the initial test data is generated randomly. There are two common problems with the random strategy. Firstly, the code coverage achieved by the initial test data could be low. Secondly, the initial test data could cause runtime errors. In contrast, **CONAGENT** generates a set of test data by creating an agent generator. The usage of powerful LLMs could reduce

Backstory

Expert Profile: Software testing and source code analysis specialist with extensive knowledge in C++ and Qt Framework, experienced in automated testing, control flow analysis (CFG), and path constraint solving.

Chain-of-thought:

1. Analyze the focal method
2. Analyze the invoked functions as follows:
 - Their individual functionality and expected output ranges, and
 - How do their outputs depend on their respective inputs, and
 - How are the inputs of these functions generated within the focal method.This step ensures that the test data satisfy the desired test path, even when influenced by function calls.
3. Generate test paths: Build a control flow graph (CFG) → Identify the minimal set of test paths, covering all statements.
4. Generate test data: For each test path, generate a test data. For each Qt project, the constructor could be a no-argument constructor or a parameterized constructor. For each primitive parameter, the value should be diverse in the valid range of its data type.
5. Format output: The format of the output must be JSON format with name of the variable and the code to initialize it.

Goal

Generate test data to maximize statement coverage for a focal method or a specific test path within the focal method.

a. Initialization prompt

Task: Generate a set of test data.

{Focal method}

{List of invoked functions}

{Few-shot examples}

b. Simplification prompt

Task: Generate a set of test data.

{Minimalistic test path}

Here are the names of the variables that need initialization, along with stub variables and their respective data types (in parentheses): ...

{Few-shot examples}

Figure 1: Format of prompt for the agent-based initialization phase (initialization prompt) and the agent-based simplification phase (simplification prompt).

the probability of runtime errors and the problem of low coverage. By using Chain-of-Thought technique, LLMs are able to know how to generate test data effectively.

4.2. Agent-based simplification phase

If the agent-based initialization phase could not achieve 100% statement coverage, the agent-based simplification phase is performed. For unvisited statements, **CONAGENT** collects a set of unexplored test paths by using depth first search technique on CFG. A test path is unexplored if

it contain at least one unvisited statement. After that, for each unexplored test path, **CONAGENT** generates test data in agent-based simplification phase. The agent-based simplification phase employs a simplification prompt, then uses an agent generator to find out test data. The format of simplification prompt is shown in Fig. 1b. To solve the *path constraint solving*, **CONAGENT** use agent generator as an agent-solver instead of an SMT solver.

4.2.1. Generate minimalistic test path

This part presents a solution to generate $\{Minimalistic\ test\ path\}$, which is a part of simplification prompt. A minimalistic test path only contains the relevant statements and branches to the arguments. The detailed algorithm is outlined in Algorithm 3. The input of the algorithm is the focal method denoted by fm and a test path denoted by tp . The output is a path constraint denoted by pc that includes all conditions and statements relevant to reaching the expected test path tp .

Algorithm 3 Minimalistic test path generation

Input: fm : a focal method, tp : an unexplored test path
Output: mtp : a minimalistic test path

```

1:  $mtp \leftarrow \emptyset$ 
2:  $D \leftarrow \emptyset$ 
3: for each statement  $s \in tp$  do
4:   if  $s$  is a condition then
5:      $(con, outcome) \leftarrow \text{extractOutcome}(s)$ 
6:      $mtp \leftarrow mtp \cup \{(con, outcome)\}$ 
7:      $D \leftarrow D \cup \text{extractVariables}(con)$ 
8:   end if
9: end for
10:  $queue \leftarrow D$ 
11: while  $queue$  is not empty do
12:    $x \leftarrow \text{dequeue}(queue)$ 
13:   for each statement  $s \in fm$  in execution order do
14:     if  $s$  modifies  $x$  then
15:       Add  $s$  to  $mtp$ 
16:        $newVars \leftarrow \text{extractModifiedVars}(s)$ 
17:        $queue \leftarrow queue \cup (newVars \setminus D)$ 
18:        $D \leftarrow D \cup newVars$ 
19:     end if
20:   end for
21: end while
22: return  $mtp$ 

```

At the start of the algorithm, the path constraint pc and a list of variables denoted by D are initialized as empty (line 1, 2). For each statement s in the test path (line 3, 9), if the statement is a branch condition (line 4), the algorithm extracts the condition and the expected outcome needed to reach the test path (line 5) and adds them to the path constraint (line 6). All variables appearing in the condition are added to D (line 7). The algorithm then creates a queue and adds D to it (line 10). It repeatedly obtains an element from the queue (line 12) and, for each statement in the source code (line 13, 20), checks if the statement modifies the value or attribute of the element (line 14). If so, the statement is added to the path constraint (line 15). All new variables

appearing in it are added to both D and the queue (line 16-18). The algorithm terminates and returns the path constraint once the queue is empty.

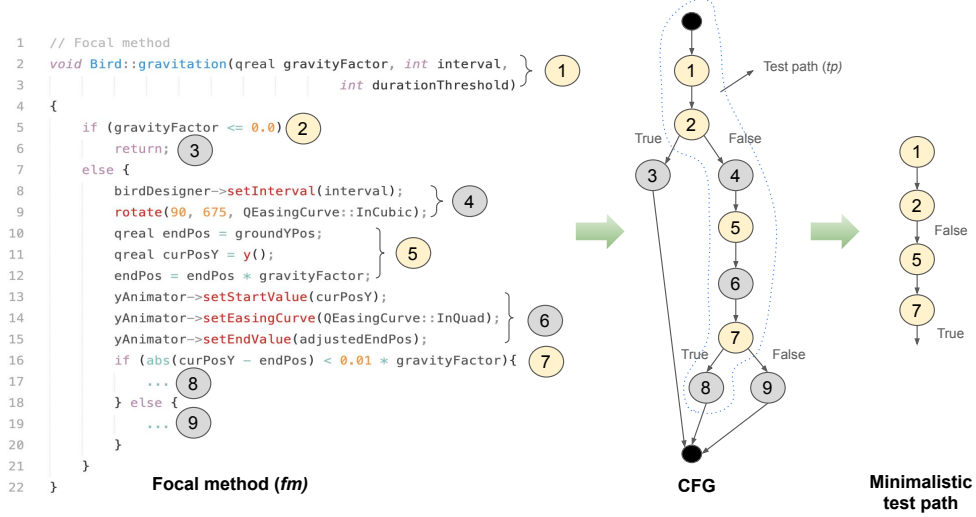


Figure 2: Example of minimalistic test path generation.

Figure 2 shows an example of minimalistic test path. The input includes the focal method `gravitation(.)` and the test path tp $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow \dots$. By applying Algorithm 3, the minimalistic test path mtp is $1 \rightarrow 2 \rightarrow 5 \rightarrow 7$. Nodes 4, 6 and 8 are eliminated because they do not influence the execution of the test path. Because the mtp is simpler than tp , agent generator can understand the simplification prompt effectively.

4.2.2. Stub invoked functions in minimalistic test path

To handle invoked functions, when stub values not matching real function outputs as describe in the problem of *stub usage*, **CONAGENT** uses an agent generator to find test data for the minimalistic test path and generate stubs. Instead of blindly assigning stub values, **CONAGENT** extracts and the agent generator analyzes the source code of the functions appearing within the minimalistic test path. This allows to generate stub values that fall within the actual output domain of the invoked functions. Agent generator considers the output domain before generating test data for the minimalistic test path.

4.3. Multi-agent phase

In case of agent-based initialization phase and agent-based simplification phase are unable to generate test data, multi-agent phase is employed. Multi-agent phase employs two agents, including agent generator and agent validator. Figure 3 illustrates the two agents and their interactive behavior with each other and testing scenarios. The agent generator solves path constraint and generates test data. Meanwhile, the agent validator provides feedback to the agent generator to improve the quality of subsequent test data. These two agents leverage their respective

knowledge bases, execution and coverage information, as well as static analysis to iteratively refine the generated test data, thereby achieving higher statement coverage while ensuring that the program remains error-free. In addition to interacting with each other, the two agents observe the outcomes of the testing process to comprehend the current testing context, thereby enabling them to take appropriate actions.

To better simulate real-world human collaboration in generating advanced test data, **CONAGENT** defines testing scenarios and facilitates interaction between the two agents. **CONAGENT** defines two primary tasks based on execution feedback, including *fix-bug task* and *enhance-coverage task*. Fix-bug task focuses on refining test data to correct issues identified during execution. Enhance-coverage task focuses on generating advanced test data to satisfy the unresolved path constraints of symbolic execution.

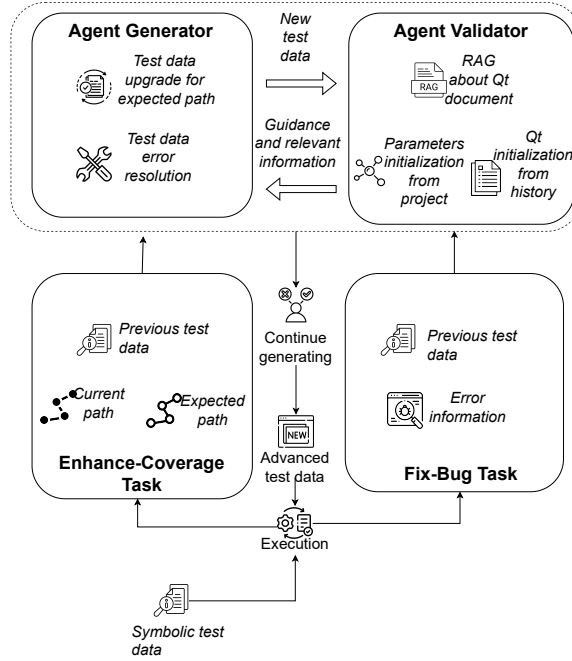


Figure 3: The overview of the multi-agent phase.

4.3.1. Fix-bug task

The Fix-bug task is triggered when the test driver fails to compile due to issues like syntax errors, version incompatibility, invalid inputs, or faults in the focal method. The agents receive comprehensive information, including the focal method’s source code, path constraints, test data, and error logs detailing the error type, cause, and line number. To effectively address this task, the agent validator is equipped with essential knowledge resources. The agent validator employs function calling to invoke a dedicated tool that efficiently retrieves relevant information from Retrieval-Augmented Generation (RAG) queries on the official Qt documentation, ensuring faster and more accurate data extraction. These documents are authoritative and ensure a

high level of accuracy regarding the Qt Framework. Additionally, a curated data of commonly encountered instantiation errors and their resolutions is maintained. Errors are systematically categorized to enable quick retrieval based on error log messages. Furthermore, past successful instantiations and execution traces from previous test drivers are stored, allowing the agent to learn correct declaration patterns for different data types. This historical reference, derived from projects where the methodology has been previously applied, significantly enhances error-handling capabilities compared to solely relying on LLM-based web searches.

Importantly, there are cases where the actual program contains logic errors, and the test data is not the root cause. Unlike LLM-based approaches that rely on a fixed number of iterations, the two-agent system collaboratively determines when to terminate test data generation after detecting fundamental program errors — such as incorrect logic or missing imports. This approach optimizes computational efficiency, reducing costs associated with invoking the Agentic framework while improving error diagnosis.

4.3.2. Enhance-coverage task

The enhance-coverage task is applied when the generated test data does not contain errors but fails to improve coverage. In other LLM-based approaches, feedback provided to the LLM is often limited to indicating which parts of the source code remain uncovered. In **CONAGENT**, however, beyond focusing on a single test path, comprehensive execution information is supplied to the Agent to understand how the existing test data behaves and which path constraints remain unsatisfied. This is achieved by comparing actual test paths with the expected ones. Consequently, the Agent can appropriately modify ineffective test data or further refine test data that has increased statement coverage but has not yet achieved full coverage.

However, for objects belonging to Qt data types or those constructed from Qt data, generating them while satisfying path constraints remains challenging due to limited real-world data. Meanwhile, an invaluable and highly relevant source of information exists within the very project under unit testing. Specifically, whenever another function calls the focal method, it must initialize the focal method’s parameters before passing them in. From this, **CONAGENT** can infer how developers initialize these parameters and identify value assignments that satisfy different test paths in the focal method. But to eliminate redundant information, only the portion of these calling functions from the start of the function to the line invoking the focal method is extracted, as this is sufficient to capture parameter initialization details. These calling functions are identified using a function dependency graph, built with an algorithm outlined in the concrete test data generation phase.

As with the fix-bug task, the discussion between the two agents determines whether further test data generation should continue. This decision is made after iterating for a predefined number of cycles, during which statement coverage may decrease or stagnate, and newly generated test data fails to provide substantive improvements. The agent validator assesses previous test data executions, provides feedback, and directs the agent generator by supplying necessary information. The agent generator then produces new test data, which the agent validator evaluates to determine whether it resolves path constraints and improves upon prior test data before responding with further guidance. At the conclusion of this iterative process, either an advanced test data set is generated, or a decision is made to halt further test data generation. In some cases, when an infeasible path constraint is identified, the two agents can provide an explanation and terminate the test data generation.

5. Empirical evaluation

To demonstrate the effectiveness of **CONAGENT**, the experiment is conducted on three real-world Qt projects. The experiment is performed on Ubuntu 22.04.5 LTS, Intel® Core™ i5-10400 CPU @ 2.90GHz × 12, 12GBs RAM. The comparable method is UT4UQ [31]. To the best of our knowledge, UT4UQ is the first concolic testing method for Qt projects. Specifically, we seek to answer the following research questions:

- **RQ1 - Statement Coverage:** Does **CONAGENT** achieve higher statement coverage than UT4UQ?
- **RQ2 - Performance:** Can **CONAGENT** generate high-quality test data with low computational cost?

5.1. Experimental setup

5.1.1. Method

Because the source code of UT4UQ is not available, the experiment implements the idea of UT4UQ using Java language. To implement agents of the **CONAGENT** methodology, the experiment leverages the CrewAI framework [49]. The proposal tool uses two popular models including GPT-4o and DeepSeek 32B. GPT-4o has good contextual understanding capabilities. However, GPT-4o cannot be installed locally and must send prompts via API to a third party. This prompt contains information about the focal method and its context. This raises concerns about security issues regarding the source code. Therefore, the experiment is conducted with DeepSeek 32B. This model can be deployed internally within the enterprise, thereby increasing the security of the project being tested.

5.1.2. Dataset

The experiment is conducted on three Qt projects as follows.

- **Flappy bird**¹ (9,059 LOC and 10 stars) : A multiresolution supported clone of Flappy Bird made with C++ and Qt Framework.
- **QtQuickExample**² (20,006 LOC and 628 stars): A project utilizes Qt and the QtQuick package to develop an interface for managing orders and payments on Youzan Mall, an e-Commerce support platform.
- **Qt360**³ (143,898 LOC and 314 stars): A project simulates the user interface of 360 10.0 software, built with Qt5. This is the largest project in our experiment, which is also the project on which the research team focus the most.

5.2. Results

5.2.1. Answer to RQ1 - Statement coverage

Table 1 presents a comparison of statement coverage between **CONAGENT** and UT4UQ. As it can be seen, **CONAGENT** outperforms UT4UQ. In terms of average statement coverage, while UT4UQ achieves 46.5%, the proposed method reaches up to above 70% for both DeepSeek 32B and GPT-4o.

¹<https://github.com/fuzun/Flappy-Bird-Qt>

²<https://github.com/toby20130333/QtQuickExample>

³<https://github.com/jun-zhang/Qt360>

Table 1: Statement coverage comparison between **CONAGENT** and UT4UQ. Columns *LOC* and *Unit* are the lines of code and the number of focal methods, respectively. Notation $\blacktriangle$ implies an increase. Column *Cov* is statement coverage. Notation $\blacktriangledown$ corresponds to a *decrease*. Column *Diff* is the difference between statement coverage (ours) and UT4UQ.

Project	File (.cpp)	LOC	Unit	UT4UQ (%)	CONAGENT (GPT-4o)		CONAGENT (DeepSeek)	
					Cov (%)	Diff (%)	Cov (%)	Diff (%)
QtQuickExample	youzanapi	58	5	20	40	$\blacktriangle 20$	40	$\blacktriangle 20$
	YouZanObject	97	9	70	77.87	$\blacktriangle 7.87$	81.57	$\blacktriangle 11.57$
Flappy bird	Button	60	6	70	100	$\blacktriangle 30$	100	$\blacktriangle 30$
	Physics	289	7	71	95	$\blacktriangle 24$	95	$\blacktriangle 24$
	Game	259	16	95	95	0	95	0
	Sound	9	1	50	50	0	50	0
	View	64	1	100	100	0	75	$\blacktriangledown 25$
	StrobeDialog	46	4	100	100	0	100	0
	MainWindow	188	4	77	77	0	77	0
	Bird	151	7	83	100	$\blacktriangle 17$	100	$\blacktriangle 17$
	Scene	455	6	45	82	$\blacktriangle 37$	68	$\blacktriangle 23$
Qt360	ScoreWidget	56	3	19	67	$\blacktriangle 48$	67	$\blacktriangle 48$
	ButtonLabel	57	6	70	77.87	$\blacktriangle 7.87$	77.87	$\blacktriangle 7.87$
	BackGroundButton	119	9	28.39	43	$\blacktriangle 14.61$	47	$\blacktriangle 18.61$
	VideoWidget	104	2	70	90	$\blacktriangle 20$	90	$\blacktriangle 20$
	OpacityWidget	20	2	25	50	$\blacktriangle 25$	50	$\blacktriangle 25$
	ScanButtonWidget	41	5	90	100	$\blacktriangle 10$	100	$\blacktriangle 10$
	youhuoButton	80	8	25	50	$\blacktriangle 25$	50	$\blacktriangle 25$
	MainscoreWidget	92	4	63	75	$\blacktriangle 12$	50	$\blacktriangledown 13$
	TopBottomWidget	49	1	0	100	$\blacktriangle 100$	100	$\blacktriangle 100$
	DynamicScaleWidget	69	2	2.18	50	$\blacktriangle 47.82$	50	$\blacktriangle 47.82$
	SafeButtonWidget	85	6	0	55.56	$\blacktriangle 55.56$	55.56	$\blacktriangle 55.56$
	SpeedoutButton	114	7	21	57	$\blacktriangle 36$	57	$\blacktriangle 36$
	AnimButtongroup...	55	4	73	61	$\blacktriangledown 12$	61	$\blacktriangledown 12$
	NumAnimWidget	85	3	7.20	57.6	$\blacktriangle 50.4$	57.6	$\blacktriangle 50.4$
	StaticButton	146	10	13	52	$\blacktriangle 39$	54	$\blacktriangle 41$
	DynamicWidget	92	2	3	53	$\blacktriangle 50$	53	$\blacktriangle 50$
	AnimButtonWidget	112	6	29.17	77.78	$\blacktriangle 48.61$	77.78	$\blacktriangle 48.61$
	NumberAnimWidget	80	5	41	65	$\blacktriangle 24$	65	$\blacktriangle 24$
	StaticTextButton	28	2	75	100	$\blacktriangle 25$	100	$\blacktriangle 25$
	DynamicButton	212	11	32	70	$\blacktriangle 38$	70	$\blacktriangle 38$
	CleanDynamicButton...	137	11	32	55	$\blacktriangle 23$	55	$\blacktriangle 23$
	StackStatusWidget	48	3	72	100	$\blacktriangle 28$	100	$\blacktriangle 28$
	WenlibackWidget	56	5	37.5	75	$\blacktriangle 37.5$	75	$\blacktriangle 37.5$
	WenliWidget	91	4	18.64	51.82	$\blacktriangle 33.18$	51.82	$\blacktriangle 33.18$
	<i>Average</i>			46.5	72	$\blacktriangle 25.5$	70.7	$\blacktriangle 24.2$

For QtQuickExample, **CONAGENT** achieves better statement coverage than UT4UQ. UT4UQ shows strong performance on YouZanObject.cpp (70%) but struggles on youzanapi.cpp with just 20% coverage. In contrast, **CONAGENT**, including both GPT-4o and DeepSeek 32B, significantly improves coverage of youzanapi.cpp, reaching 40%, double that of UT4UQ. On YouZanObject.cpp, DeepSeek 32B leads with 81.57% coverage, slightly higher than GPT-4o’s 77.87%.

This comparison highlights **CONAGENT**'s more consistent and robust coverage across different files.

For Flappy bird, **CONAGENT** demonstrates high consistency, with both models achieving 100% accuracy for most files, including Button.cpp, View.cpp, StrobeDialog.cpp, and Bird.cpp. This indicates that these files may have simpler or more modular structures, making them easier for both models to process. Notably, Game.cpp also maintains high accuracy (95%) across both models. However, some files show variations in performance between the models. For instance, Physics.cpp has a slight difference, with GPT-4o performing slightly better (100%) compared to DeepSeek (95%). A more pronounced gap appears in Scene.cpp, where GPT-4o achieves 82% while DeepSeek only reaches 68%. This discrepancy suggests that DeepSeek may struggle with files that have more complex or dense structures.

For Qt360, the increase in statement coverage is the most common trend among these cases. The extent of improvement depends on the length and complexity of the test function. For instance, DynamicWidget.cpp and VideoWidget.cpp, with an average function length of nearly 50 lines of code, show a approximately 20% increase in statement coverage compared to other methods. In contrast, shorter files such as AnimButtonWidget.cpp and WenlibackWidget.cpp with approximately 20 lines of code, witness statement coverage improvements of up to 50%. This is because longer and more complex test functions require a larger input context, which negatively impacts the AI agent's performance in generating high statement coverage test data.

Distribution of statement coverage. Given the large file size and number of files in Qt360, it is difficult to directly observe whether the statement coverage improvement is as stable as in FlappyBird or QtQuickExample. Therefore, Fig. 4 illustrates the distribution of statement coverage within the Qt360 project. UT4UQ is the least reliable, with a low median, high variability, and a right-skewed distribution, which means it struggles with consistency. Meanwhile, DeepSeek 32B is the most stable, showing a high median and compact interquartile range, indicating consistent and reliable performance. GPT-4o performs similarly, but has slightly more variability, making it a strong but more adaptable alternative.

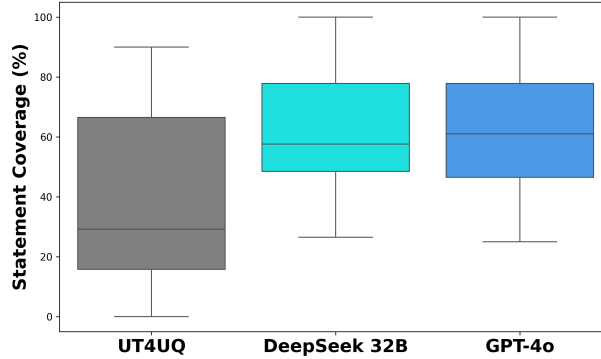


Figure 4: Statement coverage distribution of Qt360.

In summary, UT4UQ lags behind, while DeepSeek and GPT-4o deliver strong results, with DeepSeek offering greater stability and GPT-4o being slightly more flexible. This indicates that **CONAGENT** is superior to UT4UQ in handling more complex test input.

Analyze statistical meaning. To determine the statistical meaning of the result, the Wilcoxon signed-rank test is performed. There are two major reasons for that choice of inspection method:

- Wilcoxon signed rank test is often applied for data that does not follow normality. However, the number of samples in test output is a small number (< 30). Therefore, normality test is unreliable and Wilcoxon is a safer choice.
- Despite the improvement in consistency of test data generation, there are still outliers which can greatly affect the inspection method like the t-test. Wilcoxon signed rank test, otherwise, is much insensitive to outliers.

Table 2 shows the result of the Wilcoxon signed-rank test performed on Qt360. In the Wilcoxon signed rank test, the p-value determines whether two data sets differ significantly. A small p-value (≤ 0.05) indicates a significant difference, while a larger p-value (> 0.05) suggests no significant difference.

Table 2: Result of Wilcoxon signed-rank test on Qt360

Comparison	p-value
UT4UQ and GPT-4o	1.91×10^{-5}
UT4UQ and DeepSeek 32B	9.54×10^{-6}
GPT-4o and DeepSeek 32B	0.6547

As shown in the table, there are significant differences between UT4UQ and both GPT-4o ($p = 1.91 \times 10^{-5}$) and DeepSeek 32B ($p = 9.54 \times 10^{-6}$), as indicated by their extremely low p-values. This suggests that UT4UQ performance differs substantially from both models, which confirms that **CONAGENT** statistically outperforms UT4UQ. However, the comparison between GPT-4o and DeepSeek 32B yields a p-value of 0.6547, indicating no statistically significant difference between them. This suggests that their performance on the Qt360 project is nearly equivalent. This finding highlights the potential of leveraging a more cost-effective model like DeepSeek 32B for future research and real-world applications.

Finding RQ1

CONAGENT statistically and consistently outperforms the UT4UQ in most cases, especially for small and medium-sized files. In particular, both models produce nearly similar results, making cost-effective options such as DeepSeek 32B promising for further research.

5.2.2. Answer to RQ2 - Performance

Table 3 shows the comparison between GPT-4o and DeepSeek 32B across three projects. Overall, there is a significant difference in execution speed and cost efficiency between two models. The proposed method could deploy locally to ensure data privacy by using pretrained models such as DeepSeek 32B, while achieving the approximate statement coverage as GPT-4o.

GPT-4o consistently outperforms DeepSeek 32B in execution speed, completing tasks up to five times faster. For example, in the Qt360 project, GPT-4o finishes in 2 minutes and 11 seconds, while DeepSeek 32B takes 10 minutes and 24 seconds. In terms computational cost, despite using fewer tokens, GPT-4o incurs significantly higher costs, about 20 to 70 times more than DeepSeek 32B. These contradictory trends and efficiency gaps can be attributed to three key factors:

Table 3: Performance on Flappy bird, QtQuickExample and Qt360

Project	Execution Time		Computational Cost	
	GPT-4o	DeepSeek 32B	GPT-4o	DeepSeek 32B
Flappy bird	1m36s	7m48s	0.15M/2.6\$	0.19M/0.05\$
QtQuickExample	1m08s	5m06s	0.26M/4.4\$	0.32M/0.08\$
Qt360	2m11s	10m24s	0.01M/0.22\$	0.05M/0.01\$
<i>Average</i>	1m38s	7m46s	0.14M/2.41\$	0.19M/0.05\$

- Processing time: DeepSeek 32B, as a reasoning model, requires more time for reasoning phase, which increase the overall time of computation.
- Model size: GPT-4o likely has a much larger number of parameters, possibly exceeding the 175 billion parameters of GPT-3, while DeepSeek 32B has only 32 billion parameters.
- Cost: The cost of using DeepSeek 32B is intrinsically lower. Both models charges based on how many tokens you use. For GPT-4o, The cost for input tokens is \$2.50 per million, while cached input tokens are cheaper at \$1.25 per million. Output tokens, which include the model’s responses, are priced at \$10.00 per million. For DeepSeek 32B, input tokens cost \$0.12 per million, while output tokens are priced at \$0.18 per million.

DeepSeek 32B is well-suited for projects with strict security requirements, as it supports self-hosting and does not depend on external APIs. Furthermore, its relatively smaller model size compared to GPT-4o makes it more feasible for local deployment.

Finding RQ2

GPT-4o offers faster execution but at a higher cost, making it ideal for real-time tasks. In contrast, DeepSeek 32B is slower but more cost-effective and easier to be hosted locally, suitable for less time-sensitive and highly secure applications.

6. Related Works

6.1. Concolic Testing Approach

White-box testing combined with automated testing presents a suitable approach for embedded systems that demand high-quality source code to ensure reliable interaction with hardware. One of the prominent methods within this approach is concolic testing. This method was first introduced in the tool named DART [13], which automatically tests software by combining concrete execution and symbolic execution. A key factor in achieving high statement coverage during testing lies in the effectiveness of test data generation. In concolic testing, test data is generated in two phases: the initial test data generation phase and the directed test data generation phase. During the initial phase, concrete values are used to explore some initial test paths of the program a process known as concrete execution. Symbolic execution uses search algorithms like Depth-First Search (DFS) to explore test paths and generate directed test data by solving path constraint. This repeats until no new paths are found. However, concolic testing struggles

with external libraries, as current tools lack support for complex data types, such as those in the Qt Framework.

In the initial test data generation phase, test data is generated using concrete values with the objective of diversifying input values and covering a subset of test paths. One simple yet effective approach for generating test data in this phase is random test data generation [50, 32, 33, 51, 34]. For each data type, values are randomly assigned within valid ranges, but this is effective only for primitive types. The Qt Framework includes complex data types with non-primitive attributes, making initialization challenging. UT4UQ [31] addressed this by analyzing Qt class source code to extract constructors, selecting one at random, and recursively initializing parameters. However, some Qt types fail to meet the stopping condition because they do not have constructors that exclusively contain primitive types, and self-referential parameters can cause infinite recursion. Prioritizing default constructors solves this but limits value diversity, restricting meaningful object initialization.

In the directed test data generation phase, which is conducted during the symbolic execution stage of concolic testing, various strategies can be employed, such as static analysis [52], symbolic-sequence exploration [53], abstraction-based model checking [54], or explicit-state model checking [55]. These strategies fundamentally rely on underlying reasoning mechanisms, including theorem provers and SMT solvers. However, such mechanisms have inherent limitations affecting their applicability. A key challenge in path constraint solving is the limited data type support in most SMT solvers, which typically handle only primitive types in languages like C++. To use solvers like Z3, Qt data types must be translated into primitive equivalents. However, this is difficult due to Qt’s complex specialized types, such as GUI components.

A key challenge in symbolic execution is handling invoked functions tied to path constraints, as it is difficult to resolve these path constraints comprehensively or capture the full behavior of the invoked functions. Stubbing, a common solution, replaces these functions with stub variables mimicking return types. However, a major drawback of stubbing is that the generated stub values are designed solely to satisfy path constraints, often without considering additional conditions, leading to potentially unreliable coverage. For API functions or third-party libraries like Qt Framework, stubbing appears necessary [36]. However, the return values and behaviors of these functions typically depend on the Qt class parameter invoking them or those passed as parameters. Blindly replacing such functions with stub variables that merely satisfy path constraints, without ensuring the correctness of related Qt class parameter, can result in untrustworthy coverage. Instead, **CONAGENT** aims to generate Qt class parameter in a manner that ensures their interactions within Qt functions produce behaviors that satisfy the associated path constraints. For project-specific functions that are not part of the Qt Framework, stubbing is widely adopted through various approaches. For C++, Kim et al [56] and Huong et al [57] proposed automated stub generation approaches tailored to native code. However, these approaches share a fundamental limitation: they do not guarantee that the values generated for stub variables during path constraint solving accurately reflect the actual return values of the replaced functions. Consequently, this also can lead to an increase in coverage that is ultimately unreliable.

6.2. AI Approach

AI-driven approaches hold significant potential in generating Qt data type objects with specific attributes or ensuring that Qt Framework functions adhere to execution constraints, thereby enhancing statement coverage. However, effectively applying these approaches requires the integration of multiple techniques, including optimizing input for large language models (LLMs). Rosas et al. [37] highlight that the accuracy of LLM-generated results declines as the length of

the source code in the prompt increases. CoverUp [38] addresses this issue by supplying only the necessary context to cover previously uncovered lines. Similarly, TestArt [39] reformulates function call data to retain only the most essential details for LLM comprehension. Hits [40] further refines this approach by partitioning focal methods into smaller segments and generating test data for each. Nevertheless, these methods primarily focus on covering unvisited code snippets rather than analyzing specific test paths. As a result, LLMs must process multiple paths simultaneously while filtering out irrelevant statements from the provided code snippets.

Once the input has been optimized, the effectiveness of the generated test data in satisfying test paths remains an open question. Unlike SMT solvers such as Z3, AI-based approaches do not inherently guarantee accuracy or stability, as they rely on publicly available data, which may introduce errors. Even when static analysis simplifies path constraints during the symbolic phase, some path constraints may remain unsolved, particularly in complex systems. Moreover, despite Qt’s extensive documentation and widespread adoption, retrieving relevant data may be inaccurate or outdated. A key challenge in AI-driven approaches is hallucination, where large language models may generate fabricated responses in the absence of sufficient information, leading to reduced effectiveness. To address these challenges, LLM-based methods employ various techniques to refine test data and improve coverage. For instance, TestArt [39] injects positive prompts to mitigate suppression repetition. Hits [40] incorporates JUnit5 to correct errors during test generation. Other methods introduce counterexamples or predefined templates to facilitate debugging [39, 58]. Furthermore, to enhance coverage, studies integrate static analysis techniques to extract additional contextual information [38, 58]. To overcome some of the limitations of LLMs, AI Agents have been employed due to their ability to autonomously plan, interact with external tools, and adapt based on testing feedback—capabilities that traditional LLMs lack. These AI Agents address various challenges in software testing, such as debugging in a manner similar to human testers, extracting test context to enhance accuracy, handling flaky tests to improve reliability, and performing UI testing to ensure consistency and usability [59, 60, 61, 62, 63].

7. Conclusion

This paper proposed **CONAGENT** as a novel approach to improving automated testing for Qt projects. The general idea is to combine concolic testing and AI agents to generate test data to maximize statement coverage. Briefly, **CONAGENT** consists of three phases, including agent-based initialization phase, agent-based simplification phase, and multi-agent phase. Agent-based initialization phase constructs an initialization prompt, then use agent generator to generate a minimum set of test data. For each test path covering unvisited statements, agent-based simplification phase creates a simplification prompt and applies agent generator generate the next test data. If agent-based simplification phase fails, multi-agent phase combines agent generator and agent validator to generate test data more effectively. Our evaluation on three Qt open-source projects (170,000+ LOC). In terms of average statement coverage, while UT4UQ achieves 46.5%, the proposed method reaches up to above 70% for both DeepSeek 32B and GPT-4o. In terms of execution time, the proposed method could deploy locally to ensure data privacy by using pretrained models such as DeepSeek 32B, while achieving the approximate statement coverage as GPT-4o.

These findings highlight the potential application of **CONAGENT** for practical adoption in large-scale software testing. Although **CONAGENT** involves a trade-off in execution time, it achieves a consistent increase in average statement coverage across all experimental projects. We believe that **CONAGENT** has the potential to significantly enhance the current state of the field,

and we are excited about the opportunities it opens up for future research and development. By continuously refining and improving **CONAGENT**, we hope to contribute to the ongoing efforts to advance automated testing methods.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

Dat Nguyen Trong was funded by the Master, PhD Scholarship Programme of Vingroup Innovation Foundation (VINIF), code VINIF.2024.ThS.03.

CRediT authorship contribution statement

Dat Nguyen Trong: Conceptualization, Investigation, Software, Writing – original draft, Writing – review and editing. **Duc Anh Nguyen:** Conceptualization, Investigation, Methodology, Supervision, Writing – original draft, Writing – review and editing. **Pham Ngoc Hung:** Methodology, Supervision, Writing – review and editing. **Linh Ngoc Truc Tran:** Software, Writing – review and editing. **Duc Nguyen Huu:** Conceptualization, Investigation, Software, Writing – original draft.

Data availability

The source code datasets are publicly available from GitHub repositories as listed in the references. The experimental data supporting the conclusions of this article are included within the article.

References

- [1] Q. G. Oyj, Qt group oyj managers' transactions, <https://www.qt.io/stock/qt-group-oyj-managers-transactions-1491998400000>, accessed: 2025-06-24 (2025).
- [2] M. Gabrielsson, V. H. Kirpalani, *Handbook of Research on Born Globals*, Google Books, 2012, accessed: 2025-06-14. URL https://books.google.com.vn/books?id=t6uQNbVYoJoC&pg=PA172&redir_esc=y#v=onepage&q&f=false
- [3] B. Beizer, *Black-box testing: techniques for functional testing of software and systems*, John Wiley & Sons, Inc., 1995.
- [4] M. E. Khan, et al., Different approaches to black box testing technique for finding errors, *International Journal of Software Engineering & Applications* 2 (4) (2011) 31.
- [5] T. Ostrand, White-box testing, *Encyclopedia of Software Engineering* (2002).
- [6] M. E. Khan, et al., Different approaches to white box testing technique for finding errors, *International Journal of Software Engineering and Its Applications* 5 (3) (2011) 1–14.
- [7] M. Kumar, S. K. Singh, R. Dwivedi, et al., A comparative study of black box testing and white box testing techniques, *International Journal of Advance Research in Computer Science and Management Studies* 3 (10) (2015).
- [8] Valgrind Developers, Valgrind: An instrumentation framework for building dynamic analysis tools, <http://valgrind.org>, version 3.23.0 (2023).
- [9] S. Kagstrom, Kcov: Code coverage for compiled programs, <https://github.com/SimonKagstrom/kcov>, version 42 (2025).

- [10] Vector Informatik GmbH, Vectorcast: Automated testing for embedded software, <https://www.vector.com/int/en/products/products-a-z/software/vectorcast/>, software (2025).
- [11] E. Larson, T. Austin, High Coverage Detection of Input-Related Security Facults, USENIX Association, USA, 2003, p. 9.
- [12] K. Sen, Concolic testing, in: Proceedings of the Twenty-Second IEEE/ACM International Conference on Automated Software Engineering, ASE '07, Association for Computing Machinery, New York, NY, USA, 2007, p. 571–572. doi:10.1145/1321631.1321746.
- [13] P. Godefroid, N. Klarlund, K. Sen, Dart: Directed automated random testing, in: Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '05, Association for Computing Machinery, New York, NY, USA, 2005, p. 213–223. doi:10.1145/1065010.1065036.
- [14] C. Cadar, D. Engler, Execution generated test cases: How to make systems code crash itself, in: P. Godefroid (Ed.), Model Checking Software, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 2–23.
- [15] R. Majumdar, K. Sen, Hybrid concolic testing, in: Proceedings of the 29th International Conference on Software Engineering, ICSE '07, IEEE Computer Society, USA, 2007, p. 416–426. doi:10.1109/ICSE.2007.41.
- [16] K. Sen, Concolic testing and constraint satisfaction, in: K. A. Sakallah, L. Simon (Eds.), Theory and Applications of Satisfiability Testing - SAT 2011, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 3–4.
- [17] X. Wang, J. Sun, Z. Chen, P. Zhang, J. Wang, Y. Lin, Towards optimal concolic testing, in: Proceedings of the 40th International Conference on Software Engineering, ICSE '18, Association for Computing Machinery, New York, NY, USA, 2018, p. 291–302. doi:10.1145/3180155.3180177.
- [18] M. Kim, Y. Kim, G. Rothermel, A scalable distributed concolic testing approach: An empirical evaluation, in: Proceedings of the 2012 IEEE Fifth International Conference on Software Testing, Verification and Validation, ICST '12, IEEE Computer Society, USA, 2012, p. 340–349. doi:10.1109/ICST.2012.114.
- [19] L. De Moura, N. Björner, Z3: an efficient smt solver, in: Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08, Springer-Verlag, Berlin, Heidelberg, 2008, p. 337–340.
- [20] J. Christ, J. Hoenicke, A. Nutz, Smtinterpol: an interpolating smt solver, 2012. doi:10.1007/978-3-642-31759-0_19.
- [21] Y. Kim, D. Lee, J. Baek, M. Kim, Concolic testing for high test coverage and reduced human effort in automotive industry, in: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2019, pp. 151–160. doi:10.1109/ICSE-SEIP.2019.00024.
- [22] C. Cadar, D. Dunbar, D. Engler, Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs, in: Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, OSDI'08, USENIX Association, USA, 2008, p. 209–224.
- [23] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, D. R. Engler, Exe: Automatically generating inputs of death, ACM Trans. Inf. Syst. Secur. 12 (2) (Dec. 2008). doi:10.1145/1455518.1455522.
- [24] B. Elkarablieh, P. Godefroid, M. Y. Levin, Precise pointer reasoning for dynamic test generation, in: Proceedings of the Eighteenth International Symposium on Software Testing and Analysis, ISSTA '09, Association for Computing Machinery, New York, NY, USA, 2009, p. 129–140. doi:10.1145/1572272.1572288.
- [25] N. Williams, B. Marre, P. Mouy, M. Roger, Pathcrawler: Automatic generation of path tests by combining static and dynamic analysis, in: M. Dal Cin, M. Kaâniche, A. Pataricza (Eds.), Dependable Computing - EDCC 5, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 281–292.
- [26] D.-A. Nguyen, T. N. Huong, H. V. Dinh, P. N. Hung, Improvements of directed automated random testing in test data generation for c++ projects, International Journal of Software Engineering and Knowledge Engineering 29 (09) (2019) 1279–1312. doi:10.1142/S0218194019500402.
- [27] Y. Lyu, P. Mishra, Scalable concolic testing of rtl models, IEEE Transactions on Computers 70 (7) (2021) 979–991. doi:10.1109/TC.2020.2997644.
- [28] D.-A. Nguyen, T. N. Huong, H. V. Dinh, P. N. Hung, Improvements of directed automated random testing in test data generation for c++ projects, International Journal of Software Engineering and Knowledge Engineering 29 (09) (2019) 1279–1312. doi:10.1142/S0218194019500402.
- [29] K. Sen, D. Marinov, G. Agha, Cute: a concolic unit testing engine for c, SIGSOFT Softw. Eng. Notes 30 (5) (2005) 263–272. doi:10.1145/1095430.1081750.
- [30] L. N. Tung, H.-V. Tran, K. N. Le, P. N. Hung, An automated test data generation method for void pointers and function pointers in c/c++ libraries and embedded projects, Information and Software Technology 145 (2022) 106821. doi:https://doi.org/10.1016/j.infsof.2022.106821.
- [31] T. A. Bui, L. N. Tung, H.-V. Tran, P. N. Hung, A method for automated test data generation for units using classes of qt framework in c++ projects, in: 2022 RIVF International Conference on Computing and Communication Technologies (RIVF), 2022, pp. 388–393. doi:10.1109/RIVF55975.2022.10013869.
- [32] C. Csallner, Y. Smaragdakis, Jcrasher: An automatic robustness tester for java, Softw. Pract. Exper. 34 (11) (2004) 1025–1050. doi:10.1002/spe.602.

- [33] J. E. Forrester, B. P. Miller, An empirical study of the robustness of windows nt applications using random testing, in: Proceedings of the 4th Conference on USENIX Windows Systems Symposium - Volume 4, WSS'00, USENIX Association, USA, 2000, p. 6.
- [34] C. Pacheco, M. D. Ernst, Eclat: Automatic generation and classification of test inputs, in: A. P. Black (Ed.), ECOOP 2005 - Object-Oriented Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 504–527.
- [35] N. Tillmann, J. De Halleux, Pex—white box test generation for .net, in: International conference on tests and proofs, Springer, 2008, pp. 134–153.
- [36] S. Tokumoto, T. Uehara, K. Munakata, H. Ishida, T. Eguchi, M. Baba, Enhancing symbolic execution to test the compatibility of re-engineered industrial software, in: 2012 19th Asia-Pacific Software Engineering Conference, Vol. 1, 2012, pp. 314–317. [doi:10.1109/APSEC.2012.102](https://doi.org/10.1109/APSEC.2012.102).
- [37] M. R. Rosas, M. T. Sanchez, R. Eigenmann, Should ai optimize your code? a comparative study of current large language models versus classical optimizing compilers (2024). [arXiv:2406.12146](https://arxiv.org/abs/2406.12146).
- [38] J. A. Pizzorno, E. D. Berger, Coverup: Coverage-guided llm-based test generation (2025). [arXiv:2403.16218](https://arxiv.org/abs/2403.16218).
- [39] S. Gu, Q. Zhang, C. Fang, F. Tian, L. Zhu, J. Zhou, Z. Chen, TestART: Improving LLM-based Unit Testing via Co-evolution of Automated Generation and Repair Iteration, arXiv e-prints (Aug. 2024). [doi:10.48550/arXiv.2408.03095](https://doi.org/10.48550/arXiv.2408.03095).
- [40] Z. Wang, K. Liu, G. Li, Z. Jin, Hits: High-coverage llm-based unit test generation via method slicing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 1258–1268. [doi:10.1145/3691620.3695501](https://doi.org/10.1145/3691620.3695501).
- [41] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, J. Yin, Chatunitest: A framework for llm-based test generation, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Association for Computing Machinery, New York, NY, USA, 2024, p. 572–576. [doi:10.1145/3663529.3663801](https://doi.org/10.1145/3663529.3663801).
- [42] K. Sen, D. Marinov, G. Agha, Cute: A concolic unit testing engine for c, in: Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering, ESEC/FSE-13, Association for Computing Machinery, New York, NY, USA, 2005, p. 263–272. [doi:10.1145/1081706.1081750](https://doi.org/10.1145/1081706.1081750).
- [43] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al., A survey on evaluation of large language models, ACM transactions on intelligent systems and technology 15 (3) (2024) 1–45.
- [44] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, A. Mian, A comprehensive overview of large language models, arXiv preprint arXiv:2307.06435 (2023).
- [45] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili, et al., A survey on large language models: Applications, challenges, limitations, and practical usage, Authorea Preprints 3 (2023).
- [46] D. B. Acharya, K. Kuppan, B. Divya, Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey, IEEE Access (2025).
- [47] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
- [48] J. Miao, C. Thongprayoon, S. Suppadungsuk, P. Krisanapan, Y. Radhakrishnan, W. Cheungpasitporn, Chain of thought utilization in large language models and application in nephrology, Medicina 60 (1) (2024). [doi:10.3390/medicina60010148](https://doi.org/10.3390/medicina60010148).
- [49] CrewAI Inc., Crewai: A framework for building multi-agent systems, <https://github.com/crewAIInc/crewAI>, accessed: 2025-04-09 (2024).
- [50] D. L. Bird, C. U. Munoz, Automatic generation of random self-checking test cases, IBM Systems Journal 22 (3) (1983) 229–245. [doi:10.1147/sj.223.0229](https://doi.org/10.1147/sj.223.0229).
- [51] A. J. Offutt, J. H. Hayes, A semantic model of program faults, SIGSOFT Softw. Eng. Notes 21 (3) (1996) 195–200. [doi:10.1145/226295.226317](https://doi.org/10.1145/226295.226317).
- [52] C. Csallner, Y. Smaragdakis, Check 'n' crash: Combining static checking and testing, in: Proceedings of the 27th International Conference on Software Engineering, ICSE '05, Association for Computing Machinery, New York, NY, USA, 2005, p. 422–431. [doi:10.1145/1062455.1062533](https://doi.org/10.1145/1062455.1062533).
- [53] T. Xie, D. Marinov, W. Schulte, D. Notkin, Symstra: A framework for generating object-oriented unit tests using symbolic execution, in: N. Halbwachs, L. D. Zuck (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 365–381.
- [54] T. Ball, Abstraction-guided test generation: A case study, Tech. Rep. MSR-TR-2003-86 (November 2003).
- [55] W. Visser, C. S. Pundefinedsundefinedreanu, S. Khurshid, Test input generation with java pathfinder, SIGSOFT Softw. Eng. Notes 29 (4) (2004) 97–107. [doi:10.1145/1013886.1007526](https://doi.org/10.1145/1013886.1007526).
- [56] Y. Kim, D. Lee, J. Baek, M. Kim, Maestro: Automated test generation framework for high test coverage and reduced human effort in automotive industry, Information and Software Technology 123 (2020) 106221. [doi:https://doi.org/10.1016/j.infsof.2019.106221](https://doi.org/10.1016/j.infsof.2019.106221).
- [57] T. N. Huong, L. H. Chung, L. N. Tung, H.-V. Tran, P. N. Hung, An automated stub method for unit testing c/c++

- projects, in: 2022 14th International Conference on Knowledge and Systems Engineering (KSE), 2022, pp. 1–6. [doi:10.1109/KSE56063.2022.9953784](https://doi.org/10.1109/KSE56063.2022.9953784).
- [58] C. Yang, J. Chen, B. Lin, J. Zhou, Z. Wang, Enhancing llm-based test generation for hard-to-cover branches via program analysis (2024). [arXiv:2404.04966](https://arxiv.org/abs/2404.04966).
 - [59] I. Bouzenia, P. Devanbu, M. Pradel, Repairagent: An autonomous, llm-based agent for program repair (2024). [arXiv:2403.17134](https://arxiv.org/abs/2403.17134).
 - [60] Y. Zhang, H. Ruan, Z. Fan, A. Roychoudhury, Autocoderover: Autonomous program improvement, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Association for Computing Machinery, New York, NY, USA, 2024, p. 1592–1604. [doi:10.1145/3650212.3680384](https://doi.org/10.1145/3650212.3680384).
 - [61] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng, Magis: Llm-based multi-agent framework for github issue resolution, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems, Vol. 37, Curran Associates, Inc., 2024, pp. 51963–51993.
 - [62] K. Ke, NIODEbugger: A Novel Approach to Repair Non-Idempotent-Outcome Tests with LLM-Based Agent , in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, Los Alamitos, CA, USA, 2025, pp. 762–762. [doi:10.1109/ICSE55347.2025.00226](https://doi.org/10.1109/ICSE55347.2025.00226).
 - [63] F. Ricca, V. Mascardi, A. Verri, Test’n’mo: a collaborative platform for human testers and intelligent monitoring agents, in: Proceedings of the 5th ACM International Workshop on Verification and Monitoring at Runtime EXecution, VORTEX 2021, Association for Computing Machinery, New York, NY, USA, 2021, p. 17–21. [doi:10.1145/3464974.3468446](https://doi.org/10.1145/3464974.3468446).

Declaration of interests

☐The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☒The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Dat Nguyen Trong reports financial support was provided by Master, PhD Scholarship Programme of Vingroup Innovation Foundation (VINIF). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.